

Web Application Development Homework Report

FastAPI Chat Application with PostgreSQL, Redis, JWT, and GitHub OAuth

Student Name	Reham Qassem
Project Folder	wad_chat_homework
Submission Date	09.April.2026

1. Project Overview

This project is a server-rendered web application built with FastAPI and Jinja2 templates. The application allows a user to register, log in, create chats, send messages to a local GGUF language model, and save the full conversation history in PostgreSQL.

The main goal of the project is to meet the homework requirements with a clean and simple solution. The project uses PostgreSQL for permanent data, Redis for refresh sessions, Alembic for migrations, and GitHub OAuth for external login.

2. Technology Stack

- Backend framework: FastAPI
- Templates / UI: Jinja2 with server-rendered HTML
- ORM: SQLAlchemy
- Database: PostgreSQL
- Migrations: Alembic
- Cache / refresh session storage: Redis
- Authentication: local login + GitHub OAuth
- Token system: JWT access token + JWT refresh token
- Password hashing: Argon2
- Local inference: llama-cpp-python with a GGUF model

3. Why This Project Uses MVC

This project follows an MVC-style structure. The models are the SQLAlchemy database models. The views are the Jinja2 HTML templates. The controllers are the FastAPI route files that receive requests and return pages or API responses. The business logic is kept in service files, so the route files stay small and clear.

This structure makes the code easier to read and maintain. It also matches the homework requirement to prefer server-rendered HTML and MVC-style organization.

4. Authentication Design

4.1 Local Registration and Login

A user can create an account with a unique login and a password. The password is not stored in plain text. It is hashed securely before saving it in PostgreSQL.

4.2 GitHub OAuth Login

The project also supports login with GitHub OAuth. When the user signs in with GitHub, the application reads the GitHub profile, creates or links a local user, and stores the provider identity in the `oauth_identities` table.

4.3 JWT Access Token, Refresh Token, and Redis

The application uses two tokens. The access token is short-lived and is used to protect private routes. The refresh token lasts longer and is used to create a new access token when needed. Refresh sessions are stored in Redis with a 30-day TTL, so old sessions expire automatically.

5. API Structure

The project separates public and private functionality into small route groups. Main route groups include home/health routes, authentication routes, web chat routes, and API routes for chat actions.

Area	Example Routes	Purpose
Auth	/auth/register, /auth/login, /auth/logout	Local auth and GitHub OAuth
Chats	/chats, /chats/{id}	List chats, open one chat
Messages	/api/chats/{id}/messages	Send a message and save user/assistant rows
Health	/health	Basic service check

6. Code Structure

The project is organized into clear folders. Configuration is in `app/config`, database helpers are in `app/db`, models are in `app/models`, schemas are in `app/schemas`, services are in `app/services`, controllers are in `app/controllers`, templates are in `app/templates`, and the local model engine is in `app/llm`.

7. Database Schema

The database schema is small and focused. Users own chats, and chats own messages. OAuth identities are linked to users, so one user account can be connected to GitHub.

- `docker exec -it wad_chat_homework_postgres psql -P pager=off -U postgres -d wad_chat_homework -c "\dt"`
- `docker exec -it wad_chat_homework_postgres psql -P pager=off -U postgres -d wad_chat_homework -c "\d users"`
- `docker exec -it wad_chat_homework_postgres psql -P pager=off -U postgres -d wad_chat_homework -c "\d chats"`
- `docker exec -it wad_chat_homework_postgres psql -P pager=off -U postgres -d wad_chat_homework -c "\d messages"`
- `docker exec -it wad_chat_homework_postgres psql -P pager=off -U postgres -d wad_chat_homework -c "\d oauth_identities"`

```

List of relations
Schema |      Name      | Type | Owner
-----+-----+-----+-----
public | alembic_version | table | postgres
public | chats           | table | postgres
public | messages        | table | postgres
public | oauth_identities | table | postgres
public | users           | table | postgres
(5 rows)

Table "public.users"
  Column      |      Type      | Collation | Nullable |      Default
-----+-----+-----+-----+-----
id            | integer        |           | not null | nextval('users_id_seq'::regclass)
login         | character varying(50) |           | not null |
password_hash | character varying(255) |           |          |
display_name  | character varying(100) |           |          |
is_active     | boolean        |           | not null | true
created_at    | timestamp with time zone |           | not null | CURRENT_TIMESTAMP
Indexes:
    "users_pkey" PRIMARY KEY, btree (id)
    "ix_users_login" UNIQUE, btree (login)
    "uq_users_login" UNIQUE CONSTRAINT, btree (login)
Referenced by:
    TABLE "chats" CONSTRAINT "chats_user_id_fkey" FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
    TABLE "oauth_identities" CONSTRAINT "oauth_identities_user_id_fkey" FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE

Table "public.chats"
  Column      |      Type      | Collation | Nullable |      Default
-----+-----+-----+-----+-----
id            | integer        |           | not null | nextval('chats_id_seq'::regclass)
user_id       | integer        |           | not null |
title         | character varying(150) |           | not null |
created_at    | timestamp with time zone |           | not null | CURRENT_TIMESTAMP
updated_at    | timestamp with time zone |           | not null | CURRENT_TIMESTAMP
Indexes:
    "chats_pkey" PRIMARY KEY, btree (id)
    "ix_chats_user_id" btree (user_id)
Foreign-key constraints:
    "chats_user_id_fkey" FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
Referenced by:

```

```

Referenced by:
  TABLE "messages" CONSTRAINT "messages_chat_id_fkey" FOREIGN KEY (chat_id) REFERENCES chats(id) ON DELETE CASCADE

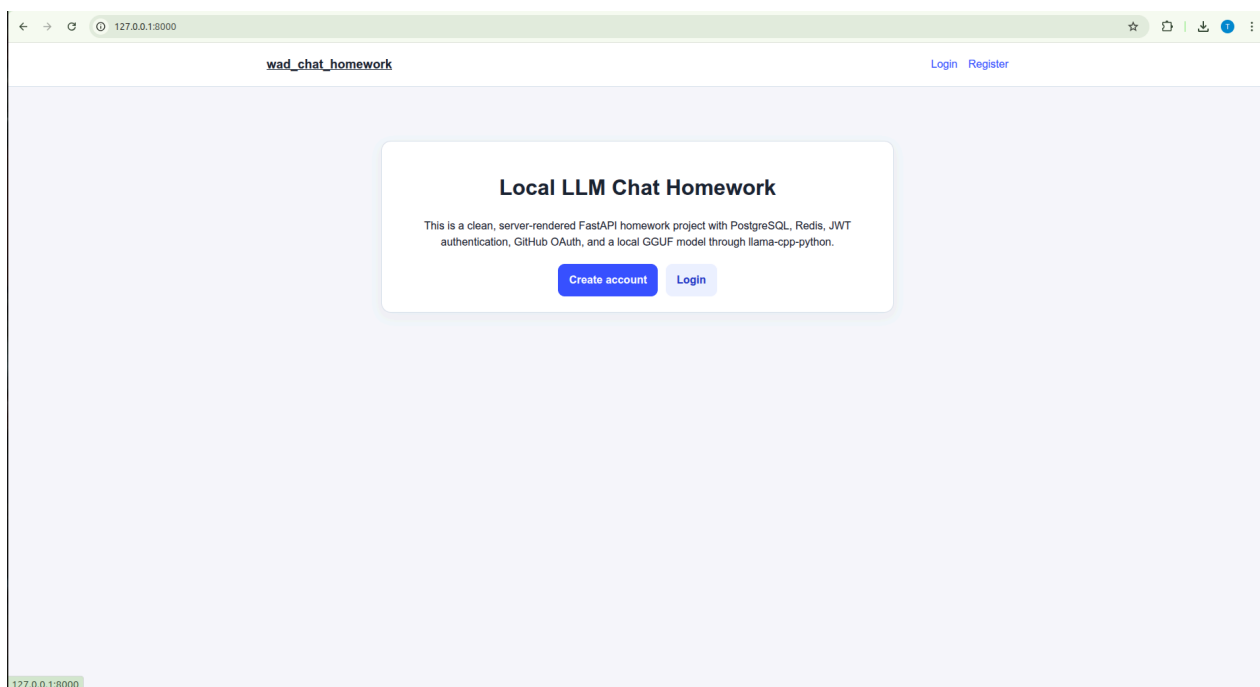
  Column |          Type          | Table "public.messages" | Collation | Nullable |          Default
  -----+-----+-----+-----+-----+-----
  id      | integer                |                          |           | not null | nextval('messages_id_seq'::regclass)
  chat_id | integer                |                          |           | not null |
  role    | character varying(20)  |                          |           | not null |
  content | text                   |                          |           | not null |
  created_at | timestamp with time zone |                          |           | not null | CURRENT_TIMESTAMP
Indexes:
  "messages_pkey" PRIMARY KEY, btree (id)
  "ix_messages_chat_id" btree (chat_id)
Foreign-key constraints:
  "messages_chat_id_fkey" FOREIGN KEY (chat_id) REFERENCES chats(id) ON DELETE CASCADE

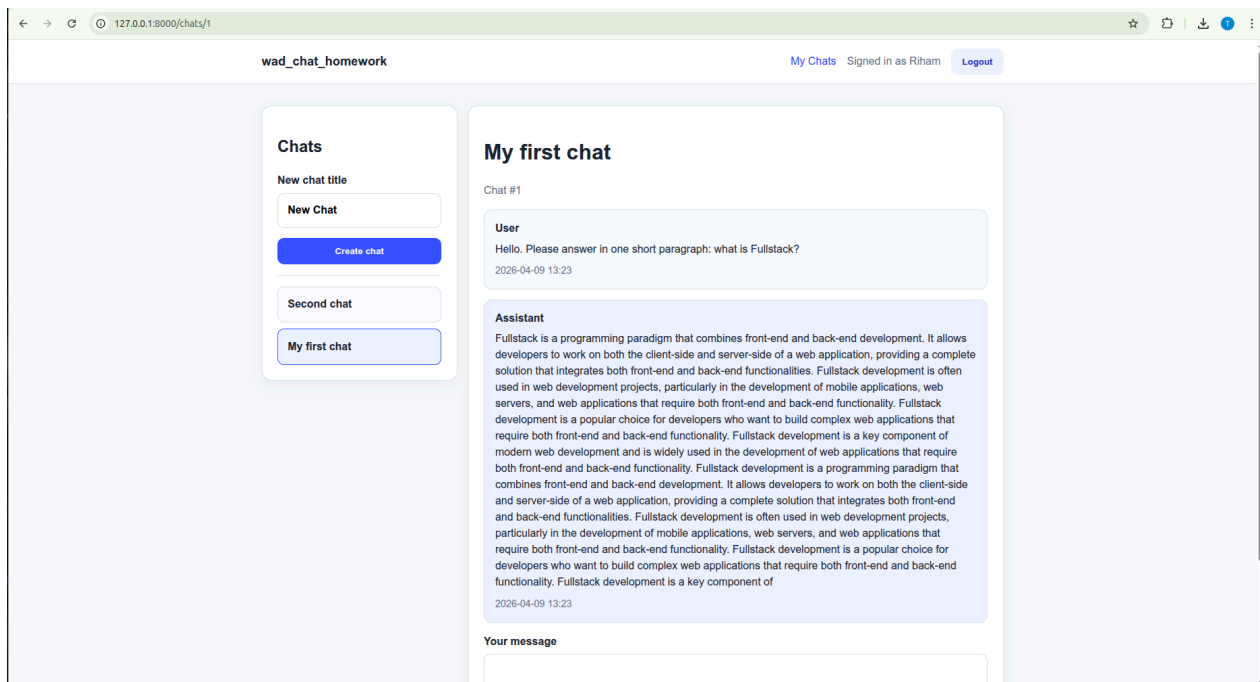
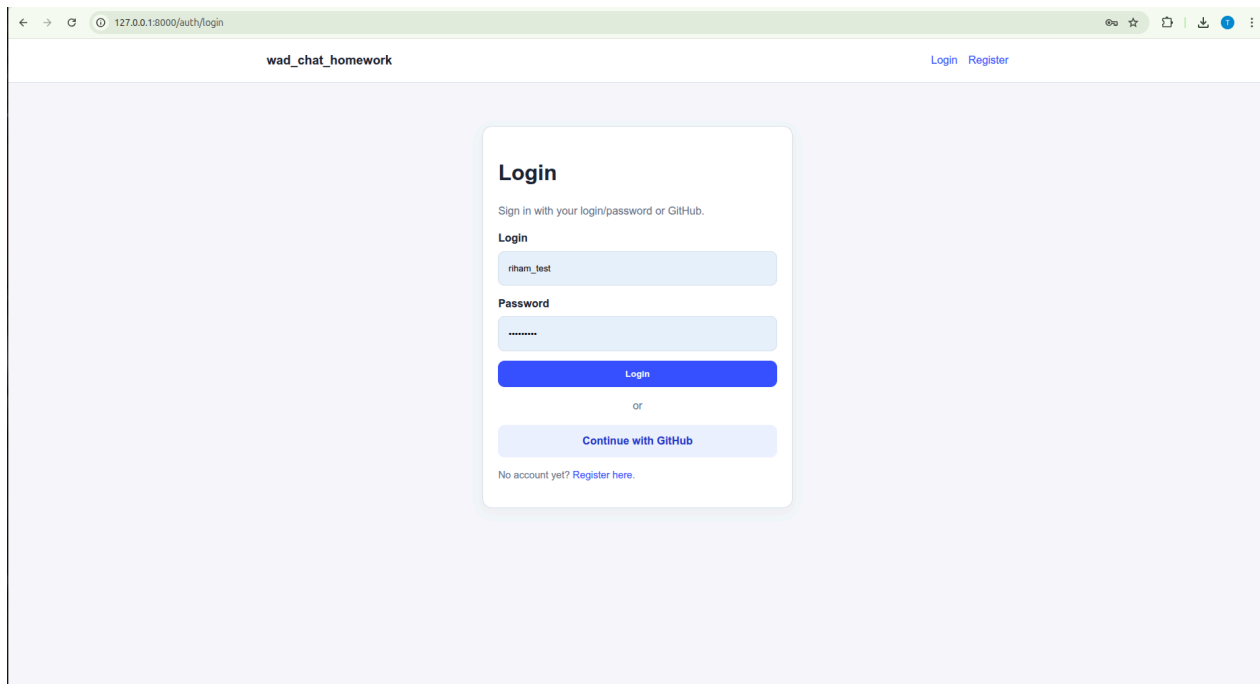
  Column |          Type          | Table "public.oauth_identities" | Collation | Nullable |          Default
  -----+-----+-----+-----+-----+-----
  id      | integer                |                          |           | not null | nextval('oauth_identities_id_seq'::regclass)
  user_id | integer                |                          |           | not null |
  provider | character varying(50)  |                          |           | not null |
  provider_user_id | character varying(255) |                          |           | not null |
  provider_login | character varying(255) |                          |           |           |
  provider_email | character varying(255) |                          |           |           |
  created_at | timestamp with time zone |                          |           | not null | CURRENT_TIMESTAMP
Indexes:
  "oauth_identities_pkey" PRIMARY KEY, btree (id)
  "uq_oauth_provider_user" UNIQUE CONSTRAINT, btree (provider, provider_user_id)
Foreign-key constraints:
  "oauth_identities_user_id_fkey" FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE

```

8. User Interface Screens

Only a small number of screenshots are included in this section, because the demo video shows the full user flow in richer detail. The screenshots below are only selected proof points for the report.





9. Testing and Evidence

The following evidence was collected during manual testing. The video contains the full flow, while the terminal screenshots below provide technical proof for database persistence, Redis TTL, and successful application behavior.

Test Item	Result / Notes
Local registration	Pass
Duplicate login blocked	Pass
Local login	Pass
GitHub OAuth login	Pass

Chat creation	Pass
Message generation with GGUF model	Pass
Message persistence after refresh	Pass
Logout and protected route check	Pass
User privacy / chat ownership check	Pass

```
(base) tharaa@tharaa-Aspire-A515-54G:~/Desktop/Riham/wad_chat_homework$ docker exec -it wad_chat_homework_postgres psql
l -P pager=off -U postgres -d wad_chat_homework -c "SELECT id, chat_id, role, LEFT(content, 80) FROM messages ORDER BY
id;"
id | chat_id | role | left
-----+-----+-----+-----
1 | 1 | user | Hello. Please answer in one short paragraph: what is Fullstack?
2 | 1 | assistant | Fullstack is a programming paradigm that combines front-end and back-end develop
3 | 2 | user | Give me 3 bullet points about FastAPI.
4 | 2 | assistant | 1. FastAPI is a micro web framework for building APIs in Python.
+
| | | | 2. It is desi
(4 rows)
(base) tharaa@tharaa-Aspire-A515-54G:~/Desktop/Riham/wad_chat_homework$
```

```
(base) tharaa@tharaa-Aspire-A515-54G:~/Desktop/Riham/wad_chat_homework$ docker exec -it wad_chat_homework_redis redis
s-cli --scan --pattern 'refresh_session:*'
"refresh_session:64fca642-3232-49ce-ba76-7db93096ecc1"
(base) tharaa@tharaa-Aspire-A515-54G:~/Desktop/Riham/wad_chat_homework$ docker exec -it wad_chat_homework_redis redis
s-cli TTL refresh_session:64fca642-3232-49ce-ba76-7db93096ecc1
(integer) 2591573
(base) tharaa@tharaa-Aspire-A515-54G:~/Desktop/Riham/wad_chat_homework$ docker exec -it wad_chat_homework_redis redis
s-cli GET refresh_session:64fca642-3232-49ce-ba76-7db93096ecc1
"{\"user_id\": 1, \"login\": \"riham_test\"}"
(base) tharaa@tharaa-Aspire-A515-54G:~/Desktop/Riham/wad_chat_homework$
```

10. Run and Setup Summary

The project runs on Ubuntu with Python, Docker, PostgreSQL, Redis, and a local GGUF model file. The model path is configured through the .env file. Database tables are created with Alembic migrations.

A separate setup guide can be attached in the submission. Paste the final repository link, video link, and any teacher-specific notes in the placeholders below.

Repository URL:

11. Conclusion

This project meets the main homework requirements: FastAPI, PostgreSQL, Alembic migrations, Redis refresh sessions, JWT authentication, GitHub OAuth, server-rendered HTML, and a chat feature connected to a local GGUF model.